Career Compass SG — Project Report

Module 1 Assignment · Group 7 · Singapore Jobs Analytics Topic: Career Recommendations

Dataset: `SGJobData.csv` — 1,048,585 MyCareersFuture job postings, October 2022 – May 2024 Tooling: Python 3 · pandas · PyArrow · Streamlit · Plotly · matplotlib / seaborn Deliverables: cleaning pipeline (`src/`) · EDA notebook (`notebooks/01_eda.ipynb`) · 7-page dashboard (`app/`)

---

1. Business Case

Scenario. A job seeker or mid-career switcher in Singapore has to choose which career track to aim at next. Job boards answer "*what jobs exist right now?*". They do not answer "*which of these is worth entering?*" — and that is the question that actually determines the next five years of someone's working life.

Objective. Help that person decide **which career track to target**, by measuring the four things that decide whether a track is worth entering, and that no job board shows side by side:

Dimension	The question it answers
Demand	Are there enough openings to realistically land one?
Pay	What does this track actually pay, at my level?
Competition	How many people am I up against for each seat?
Accessibility	Can I get in with the experience I have *today*?

Target users and value. Primarily job seekers and career switchers; secondarily the career coaches who advise them. The product ranks **215 career tracks** (43 job categories × 5 seniority bands) into a personal shortlist, with every component of the score exposed and every weight user-adjustable. A user who cares about money gets a different answer from one who needs to get hired quickly — and both answers are defensible from the same data.

Success criterion. A user should be able to say, within three minutes of opening the dashboard: "*these are the three tracks I should target, this is what they pay, and this is how contested they are.*"

Why this framing was chosen. Our first instinct was a "top paying jobs in Singapore" dashboard. The EDA killed it: pay, volume and competition turn out to be nearly independent of one another (Section 2.5). A ranking on any single dimension is actively misleading, so the product had to be multi-dimensional and user-weighted from the start.

---

2. Data Handling & Process

2.1 Tools and architecture

Python + pandas, with a deliberate three-stage split so that the dashboard never touches the raw CSV:

```
SGJobData.csv (273 MB, 1.05M rows)
        │
        ▼  src/etl_clean.py     — chunked read, cleaning, feature engineering   (~15 s)
   3 parquet files
        │
        ▼  src/build_aggregates.py — summaries + Career Fit Score components     (~2 s)
   8 aggregate tables + KPI JSON
        │
        ▼  app/ (Streamlit)     — reads parquet, filters live                (<0.1 s/interaction)
```

Parquet with categorical dtypes is what makes the dashboard feel instant: the working table is **1.77 million rows in 170 MB of memory**, and a full filter-plus-group-by cycle takes **0.06 seconds**. That is why the dashboard filters recompute live rather than serving frozen aggregates.

2.2 Loading 1M+ rows

Following the brief, everything was designed on a 50,000-row sample and only then scaled. The full file is read in **6 chunks of 200,000 rows** (`pd.read_csv(chunksize=200_000)`), each cleaned independently, then concatenated for the whole-file rules (de-duplication and outlier capping) that cannot be done chunk-by-chunk. Total runtime: **15 seconds** for all 1,048,585 rows.

2.3 Cleaning decisions — and why

Every rule below is recorded in `src/config.py` with its threshold, and the pipeline writes an audit trail to `data/processed/data_quality_report.json`.

#	Issue found	Decision	Why this and not something else
1	`occupationId` is **100% null** across all 1,048,585 rows	Drop the column	A column that is always empty cannot inform anything. Verified on the full file, not the sample.
2	`status_id` duplicates `status_jobStatus`	Drop the column	Redundant numeric encoding of a text column we already keep.
3	**3,988 rows (0.38%)** missing job id, title, company and all dates	Drop the rows	Checked first that the gaps are the *same* rows — they are wholly empty export lines, not individually missing fields. Dropping columns would have been the wrong fix.
4	`salary == 0` on 3,988 rows	Set to missing	Zero means "not disclosed", not "this job pays nothing". Treating it as a number would drag every average down. *(These turned out to be the same broken rows as #3.)*
5	**10,016 rows** with salary < $500 or > $60,000/month	Blank the salary, **keep the row**	These are annual figures in a monthly field. The posting still counts as demand — we just cannot trust its pay.
6	`salary_min > salary_max`	Swap the two values	The fields were entered the wrong way round; the information is there. Dropping would discard a real posting.
7	Long right tail on salary	**Cap** at the 1st/99th percentile ($1,150 – $16,500), **19,647 rows** touched	Capping keeps the row — so demand counts stay correct — while stopping a thin tail of executive packages from steering every average. Deleting would have biased the demand signal.
8	21 postings requiring > 40 years' experience	Set to missing	A 50-year requirement is a typo, not a job.
9	87 postings with 0 or > 500 vacancies	Floor at 1 / blank above 500	A posting is at least one seat; bulk entries above 500 are placeholders.
10	Duplicate job ids	De-duplicate on `job_id`, `keep="last"`	**Found zero genuine duplicates.** `.duplicated()` initially reported 289 in the sample — those were repeated *blanks*, not repeated ids. We kept the safeguard and documented it as a no-op rather than quietly deleting the check.
11	`categories` is a JSON array — a job holds up to 3	Parse and explode into a long table	See 2.4.

The consistent principle: blank the untrustworthy field, keep the trustworthy row. A posting with a broken salary still proves a job exists. After cleaning, **1,044,597 rows (99.6%) survive**, with salary disclosed on **99.0%** of them.

2.4 One job, many categories

`categories` arrives as `[{"id":21,"category":"Information Technology"}]` and a job carries **1.69 categories on average** (max 3). Counting rows answers "*how many postings are there*"; counting rows after exploding answers "*how many IT jobs are there*". These are different questions, so the pipeline produces **two tables**:

- `jobs_clean.parquet` — 1,044,597 rows, one per posting → market-level totals

- `jobs_by_category.parquet` — 1,767,829 rows, one per posting × category → category analysis

Category shares therefore **overlap and do not sum to 100%**, which the dashboard states explicitly. Mixing the two tables up is the single easiest way to get this dataset wrong; an integer `job_key` lets every market-level statistic de-duplicate before it counts. *(We hit this bug ourselves mid-build: the overview page briefly reported 1.66M postings instead of 1.04M.)*

2.5 Feature engineering

Raw columns describe a *posting*. A job seeker needs columns that describe an *opportunity*.

Feature	Definition	Why it exists
`seniority`	9 `positionLevels` → 5 bands (Entry / Junior / Mid / Senior / Management)	Nine levels is more than a person can reason about
`salary_band` , `experience_band`	`pd.cut` brackets	"Which bracket am I in?"
`applications_per_vacancy`	applications ÷ vacancies	**The competition metric the whole product rests on**
`apply_rate`	applications ÷ views	Of those who saw it, how many acted
`is_hard_to_fill`	`repost_count >= 1`	The employer could not fill it — that is an opening
`is_entry_friendly`	requires ≤ 1 year	Can I apply today, with what I have?
`title_clean`	strip marketing noise from the title	Count **roles**, not adverts
`skill`	50-term regex dictionary matched against titles	Which title terms carry a pay premium

Two notes on judgement calls:

- **`is_hard_to_fill` uses `repost_count >= 1`, not `>= 2`.** `metadata_repostCount` is capped at 2 in this extract (values are only 0, 1, 2), so "reposted twice" would flag just 1.4% of postings. Any repost at all is the meaningful signal: 4.1%.
- **`title_clean` matters more than it sounds.** Singapore job titles carry heavy marketing: `"Urgent Hiring!!! Business Development Manager (MES, Pre-sales) - Up to $9K"`. Without stripping bracketed skill lists, embedded salaries and urgency words, the "most common job title" list is mostly noise.

2.6 EDA highlights — the findings that shaped the design

(a) The dataset is two stitched extracts. This is the most important thing we found, and we found it as a sanity check rather than by looking for it. Plotting engagement over time:

Postings from	Mean views	Share with zero applications
Oct 2022 – Jun 2023	69 – 255	4 – 28%
Jul 2023 – May 2024	**4 – 12**	**63 – 79%**

Views per posting collapse from ~111 to ~5 exactly at the Jun/Jul 2023 boundary. This is not a collapse in interest — postings from 2023-07 onward were captured at or near posting time, so their counters never accumulated.

> **What we did.** Demand and salary use all 1.04M rows. **Competition is computed only on postings up to 2023-06-30** — 203,702 of them — and is labelled as such everywhere it appears, including a permanent caveat box on the Competition page. Had we ignored this, every career track would have looked uncontested and the recommender would have confidently pointed people at the most crowded markets in Singapore.

(b) Volume ramps up until May 2023. Monthly postings climb from 172 (Oct 2022) to a stable ~75,000/month from May 2023. That ramp is our data collection filling up, not the job market waking up, so **all trend and growth views start at May 2023** and the overview chart shades the earlier period as "partial data collection".

(c) Size, pay and competition are nearly independent — the finding that defined the product:

Category	Postings	Median salary	Applicants per seat
Information Technology	140,866	**$6,500**	4.8
Admin / Secretarial	117,854	$2,900	5.7
F&B	73,731	$3,301	**1.0**
Personal Care / Beauty	16,932	$4,000	**0.7**
Social Services	8,684	$3,500	**11.3**

Competition varies by a factor of **16** across sizeable categories. Two categories with similar openings can offer completely different odds, and nothing on a job board tells you this. A single-dimension ranking would be actively misleading — hence the four-component score.

(d) Hard-to-fill means underpaid, not elite. Reposted jobs pay a median of **$3,700** against **$3,850** for jobs filled first time. We reframed this feature as negotiation leverage rather than a prestige signal.

(e) The salary distribution is right-skewed (skew 1.95; mean $4,674 vs median $3,850). Every headline figure in the dashboard is a **median**, and the overview histogram plots both lines so the reader can see why.

(f) The top "employers" are recruitment agencies. The five largest posters are THE SUPREME HR ADVISORY (61,638 postings), RECRUITPEDIA (50,444), RECRUIT EXPRESS (33,170), ANRADUS (25,805) and RECRUIT EXPERT (21,608). `postedCompany_name` is the *poster*, not the hiring employer, so the dashboard labels the column **"Poster"** and warns the user directly.

---

3. Dashboard / App

Solution type: a multi-page **Streamlit** application with **Plotly** charts, run locally against the pre-built parquet files. Seven pages, ~17 distinct chart types, one global filter sidebar shared across every page.

3.1 The views

Page	What it answers	Charts
① **Market Overview**	How big is this market and what does it pay?	5 KPI tiles · horizontal bar (top 15 categories) · histogram with median/mean rules · donut (employment mix) · line with shaded partial-coverage period
② **Career Explorer** *(drill-down)*	What is a career in *this* category actually like?	box plots by seniority · category × seniority heatmap · horizontal bar (top titles) · ranked poster table
③ **Pay & Progression**	Where is the money, and what is a year of experience worth?	lollipop (pay ranking vs market median) · dumbbell (25th–90th percentile spread) · line with interquartile band (experience curve) · violin (employment type)
④ **Demand vs Competition**	Which tracks have volume *without* the queue?	four-quadrant bubble map (log axis, size = seats, colour = pay) · bar (repost rate) · shortlist table
⑤ **Skills**	Which title keywords carry a premium?	treemap (demand) · diverging bar (premium vs market) · scatter (premium vs accessibility)
⑥ **Career Recommender** ⭐	**Which tracks should *I* target?**	stacked contribution bars · radar (top-3 profile) · ranked evidence table · CSV download
⑦ **Trends**	What is growing and what is shrinking?	diverging growth bar · multi-line comparison · 100% stacked area (seniority mix) · line (median pay over time)

3.2 Interactivity

- **Global sidebar filters** — posting period, employment type, seniority, salary range, open-postings-only — persist across all seven pages via `st.session_state` and recompute every chart live over 1.77M rows.
- **Per-page controls** — category selector, up-to-3 comparison pickers, the recommender's four weight sliders and personal inputs.
- **Every chart has hover tooltips** carrying the supporting numbers (postings, median pay, competition) rather than only the plotted value.
- `@st.cache_data` on every load and aggregation keeps interactions under a tenth of a second.

One deliberate exception to the filters. Competition metrics ignore the period filter and always use their own reliable window. Competition is a property of a career track, not of the months a user happens to be browsing — and after Jun 2023 the counters do not exist. A user looking at 2024 postings still sees real competition figures, clearly labelled.

3.3 Design choices

- **Every chart states its finding in words above it**, in a highlighted box — "the chart is chosen by the message, not by taste" (Lesson 1.10). The chart is then built to deliver that sentence.
- **Colour is used as an argument, not decoration.** One accent colour highlights the single bar or line that carries the point; everything else is a recessive blue or grey.
- **A colour-blind-safe categorical palette** in a fixed order (the ordering *is* the safety mechanism — adjacent pairs must stay separable). Scatter and bubble charts are capped at three series, because those forms compare every pair at once and only the first three slots clear the separation threshold. That cap is why the comparison pickers are limited to three categories.

- **Sequential blue** for continuous magnitude (heatmaps, bubbles); **diverging blue↔red with a grey midpoint** for "above vs below the market".
- **Zero-based axes on every bar chart**, and the one logarithmic axis (the opportunity map, where category sizes span two orders of magnitude) is labelled as such in the axis title.
- **No dual-axis charts anywhere.**
- Medians rather than means throughout; box plots and dumbbells wherever the spread matters more than the midpoint.

3.4 How each view serves the objective

Pages ① – ⑤ are the **evidence**; page ⑥ is the **product**. The Recommender is where a user gets an answer, and every other page exists so that they can check it. A user who does not believe their shortlist can open Career Explorer and see the salary distribution, the real job titles and the hiring agencies behind any recommended track.

3.5 The Career Fit Score

Four components, each converted to a **0–100 percentile rank** across all 215 tracks so that counts, dollars and ratios become comparable:

Component	Measure	Default weight
Openings	percentile of $\log_{10}$(postings) — 40,000 postings is not "twice as good a bet" as 20,000	30%
Pay	percentile of median advertised salary	30%
Low competition	percentile of the *inverse* of applications ÷ vacancies (reliable window only)	25%
Accessibility	share of the track's postings asking for ≤ the user's years of experience, re-ranked across surviving tracks	15%

A track needs ≥ 200 postings to be scored at all. **The weights are sliders, not constants** — and the ranking genuinely responds to them:

Rank	Balanced (default)	Money first	Easiest to enter
1	Information Technology – Mid	Information Technology – Senior	Human Resources – Entry
2	Information Technology – Senior	Information Technology – Mid	Sales / Retail – Junior
3	Sales / Retail – Junior	Information Technology – Management	Sales / Retail – Entry
4	Healthcare / Pharmaceutical – Mid	Banking and Finance – Management	Education and Training – Entry
5	Sales / Retail – Entry	Engineering – Management	Events / Promotions – Entry

Easiest to enter shares **no entries at all** with the default ranking. These are genuinely different recommendations for genuinely different people, which is the entire argument for making the score user-weighted rather than fixed.

What the score is not. It knows nothing about the user beyond their years of experience, and nothing about whether they would enjoy the work. It ranks *market conditions* — one input to a career decision, not the

decision. The dashboard says so, in the app.

---

4. Key Insights

1. Information Technology is the only category that is both the largest and the best paid — 140,866 postings at a $6,500 median, against a market median of $3,850. It is also the only large category where volume and pay point the same way.

2. …but IT postings fell 11.7% over the last six months, the third-steepest decline of any category. The safest-looking track in Singapore is cooling, and a recommendation that ignored the trend would be a year out of date.

3. The best odds are not where the money is. F&B (1.0 applicants per seat), Personal Care / Beauty (0.7) and Customer Service (2.4) offer real volume with almost no queue. Social Services (11.3), Risk Management (8.7) and Advertising / Media (8.5) are the crowded ones — and Risk Management is well paid, which is exactly why it is contested.

4. Experience is worth about $900 a month per year in IT — from $3,100 at zero years to $14,000 at twelve. Across the market the ladder runs $2,675 (Entry) → $3,150 (Junior) → $4,000 (Mid) → $5,000 (Senior) → $6,500 (Management).

5. The skill premium is a factor of 4.6. An "AI / Machine Learning" title pays $8,750 (+127% vs market); "Cleaning / Housekeeping" pays $1,900 (–51%). Data Engineering (+108%), DevOps (+101%) and Java (+95%) follow. This is the clearest investment signal in the dataset.

6. Hard-to-fill roles are underpaid, not elite — $3,700 median vs $3,850. Treat a repeatedly reposted advert as negotiating leverage.

7. The seniority mix is stable (Mid 33% → 37% over the window). Good news for a career switcher: the ladder is not being pulled up.

8. Flexibility is expensive. Part-time roles pay a $2,500 median against $3,900 for permanent — a 36% discount. Contract roles, however, pay *more* ($4,250), so a contract is not automatically a pay cut.

---

5. Challenges & Learnings

Challenge 1 — a data quality trap that would have inverted our conclusions. The engagement break at Jun 2023 was invisible in summary statistics and only appeared when we plotted the counters over time. Had we trusted them, the market would have looked uncontested everywhere and the recommender would have sent users into the most crowded tracks. **Learning: plot every metric against time before you trust it**, even when you have no reason to suspect it.

Challenge 2 — the double-counting bug we shipped to ourselves. Exploding categories turns 1.04M postings into 1.77M rows. Our first overview page reported 1.66M postings and a $3,750 median — both wrong, and both plausible enough that nobody would have questioned them. Fixed by adding an integer `job_key` and routing every market-level statistic through a de-duplication step. **Learning: when one row means two different things in two tables, make the distinction structural rather than remembered.**

Challenge 3 — an unlisted trap in `.duplicated()`. It reported 289 duplicate job ids in our sample. All 289 were repeated *blanks*. **Learning: `.dropna()` before `.duplicated()`, and check what a suspicious count is**

actually made of before acting on it.

Challenge 4 — performance on 1M rows. Naïvely re-reading the CSV per interaction made the dashboard unusable. Parquet + categorical dtypes + `@st.cache_data` brought the working set to 170 MB and interactions to 0.06 s, which is what let us keep live filtering instead of falling back to pre-computed aggregates.

Challenge 5 — restraint in the charts. Our first opportunity map labelled all 43 categories and was unreadable. Cutting to nine labels made the point immediately. **Learning: a chart's job is one sentence; anything not serving that sentence is noise.**

Next steps

1. **Skills are inferred from job titles**, since the dataset has no skills field. Parsing job *descriptions* would turn a keyword proxy into a real skills taxonomy.
2. **Resolve agencies to real employers**, so "who is hiring" reflects employers rather than recruiters.
3. **A saved user profile and alerts** — the natural product step from a dashboard to a system: store the user's weights and notify them when a track's score moves.
4. **A fresher, single-pass extract** would remove the engagement-window restriction entirely and let competition be measured across the full period.

Appendix — Repository

```
Module1Group7 project/
├── SGJobData.csv                    raw data (273 MB, not committed)
├── requirements.txt
├── SETUP.md                         how to run
├── src/
│   ├── config.py                    every threshold, mapping, palette — the audit trail
│   ├── features.py                  cleaning + feature engineering
│   ├── etl_clean.py                 stage 1: chunked ETL over 1.05M rows
│   └── build_aggregates.py          stage 2: aggregates + Career Fit Score
├── notebooks/01_eda.ipynb           the EDA behind every decision above
├── data/processed/                  parquet outputs + data_quality_report.json
├── app/
│   ├── Home.py                      ① Market Overview  (dashboard entry point)
│   ├── utils.py                     loading, filters, chart styling
│   └── pages/1–6_*.py               ② – ⑦
└── reports/
    ├── PROJECT_REPORT.md            this document
    ├── PRESENTATION.md              10-minute presentation script
    └── figures/                     charts exported by the notebook
```

To run the dashboard:

```
pip install -r requirements.txt
python src/etl_clean.py && python src/build_aggregates.py   # ~17 s, needs SGJobData.csv
streamlit run app/Home.py
```

The raw CSV (273 MB) and the derived parquet (~145 MB) are not committed to the repository — the raw file exceeds GitHub's 100 MB limit and the parquet is fully reproducible from it in about 17 seconds. The two small JSON audit files in `data/processed/` *are* committed, so the cleaning record can be read without running anything. Full instructions in `SETUP.md`.